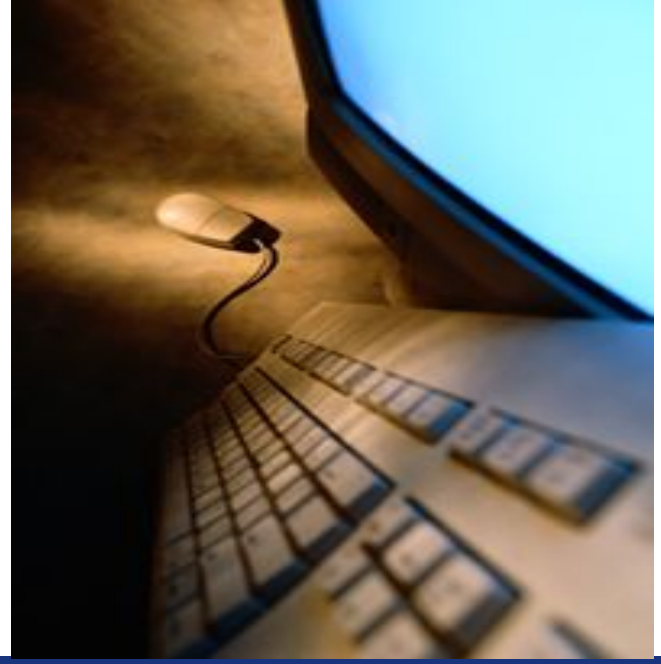
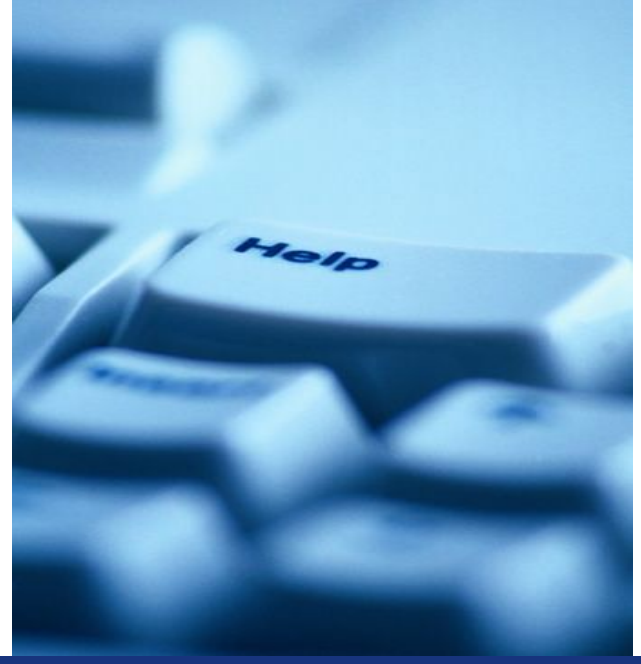
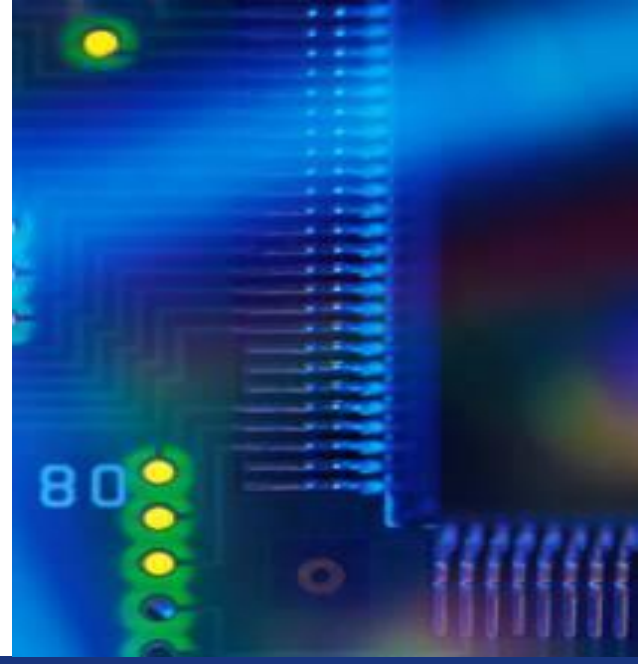
UNIVERSITAS
BUDI LUHUR







FAKULTAS TEKNOLOGI INFORMASI

Pemrograman Java Web dan Enterprise

[PG191/ 3 SKS]

Semester Gasal 2026/2027

Dosen : Riskiana Wulan S.Kom, M.Kom







Pertemuan 3
Fundamental Web, Protokol HTTP & Servlet






Tujuan Pembelajaran

- **Setelah mengikuti perkuliahan ini, mahasiswa mampu:**
  - Memahami arsitektur dasar Client-Server dan alur komunikasi protokol HTTP (Request & Response).
  - Mengidentifikasi anatomi pesan HTTP: HTTP Method, Status Code, Header, dan Body.
  - Memahami konsep Servlet Container (Apache Tomcat) serta siklus hidup Servlet (init, service, destroy).
  - Membuat Servlet pertama (`ProductServlet`) menggunakan anotasi `@WebServlet` yang menerima HTTP Request dan mengembalikan response data katalog produk.

- **Prerequisite:**
  - Mahasiswa telah menyelesaikan materi Week 2: membuat project Java menggunakan Apache Maven (`pom.xml`), mengelola data di memori menggunakan `List<Product>`, serta menangani error menggunakan try-catch dan custom exception.






Big Picture

- **Dari Program Terminal ke Aplikasi Web**
  - Program Java pada Minggu 1 dan 2 berjalan terbatas di dalam terminal komputer lokal.
  - Aplikasi web enterprise memerlukan antarmuka yang dapat diakses banyak pengguna secara bersamaan dari berbagai perangkat melalui jaringan internet.
  - Pengembangan aplikasi web Java dimulai dari komponen Servlet dan Servlet Container (Tomcat) yang menjadi pintu gerbang penerima permintaan dari web browser (Client) menuju aplikasi backend Java (Server).






Arsitektur Web & Protokol HTTP

- **Protokol HTTP mengatur komunikasi data antara Client (browser/mobile app) dan Server (aplikasi Java backend).**

- **Alur Client-Server:**

1. Client mengirimkan pesan HTTP Request ke alamat URL tertentu (misalnya `http://localhost:8080/products`).
2. Server Java menerima permintaan tersebut, memproses logika bisnis (misalnya mengambil daftar barang dari `ProductRepository`), dan mengemas hasilnya.
3. Server mengembalikan jawaban berupa pesan HTTP Response ke browser untuk ditampilkan kepada pengguna.

- **Setiap framework web Java modern (termasuk Spring Boot) dibangun di atas fondasi komunikasi HTTP Request-Response ini.**







Arsitektur Web & Protokol HTTP

Protokol HTTP mengatur komunikasi data antara Client (browser/mobile app) dan Server (aplikasi Java backend).

Client
(Browser / Mobile App)

Web Browser
(Chrome, Firefox, dll.)

Mobile App
(Android / iOS)

1 Client mengirim HTTP Request
Client mengakses URL tertentu, misalnya:

`http://localhost:8080/products`

HTTP Request

```
GET /products HTTP/1.1
Host: localhost:8080
Accept: application/json
```

2 Server memproses permintaan
Server Java menerima permintaan, memproses logika bisnis (misalnya mengambil daftar barang dari ProductRepository), dan mengemas hasilnya.

HTTP Response

3 Server mengirim HTTP Response
Server mengembalikan jawaban berupa pesan HTTP Response ke browser/mobile app agar dapat ditampilkan kepada pengguna.

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "products": [
    { "id": 1, "name": "Laptop", "price": 1000 },
    { "id": 2, "name": "Mouse", "price": 50 }
  ]
}
```

Server (Java Backend)
spring boot

Controller
(menerima HTTP request)

Service
(logika bisnis, misalnya mengambil data produk)

ProductRepository
(akses database)

Database
(misalnya PostgreSQL, MySQL, dll.)






Poin Penting

1. **HTTP adalah protokol standar** untuk komunikasi data di web.
2. **Client** (browser/mobile app) mengirimkan **HTTP Request** ke URL tertentu.
   `http://localhost:8080/products`
3. **Server Java** menerima permintaan, memproses **logika bisnis** (misalnya menganbil data dari `ProductRepository`), dan mengemas hasilnya.
4. **Server** mengembalikan jawaban berupa **HTTP Response** ke browser/mobile app untuk ditampilkan kepada pengguna.
5. **Framework web Java modern** (termasuk Spring Boot) dibangun di atas fondasi komunikasi **HTTP Request-Response** ini.

Kesimpulan

- Aplikasi web Java (termasuk Spring Boot) pada dasarnya bekerja dengan **mekanisme HTTP Request-Response**, di mana client mengirim permintaan, server memprosesnya, lalu mengirimkan respons kembali ke client.






Anatomi Protokol HTTP

- **Pesan HTTP terstruktur atas empat komponen utama:**
  - **HTTP Method**: menentukan jenis tindakan yang diminta Client.
    - GET: membaca/mengambil data (misalnya melihat katalog produk).
    - POST: mengirim dan membuat data baru (misalnya menambah produk baru).
    - PUT: memperbarui data yang sudah ada (misalnya mengedit harga produk).
    - DELETE: menghapus data dari server.
  - **Status Code**: kode respon tiga digit dari server yang menginformasikan status pemrosesan.
    - 200 OK: permintaan berhasil diproses.
    - 404 Not Found: URL/data yang diminta tidak ditemukan di server.
    - 500 Internal Server Error: terjadi kesalahan/crash logika di kode program server Java.
  - **Header**: informasi meta pendukung transaksi, seperti format data (Content-Type).
  - **Body**: payload utama yang dikirimkan, seperti data form produk baru atau respon data produk.

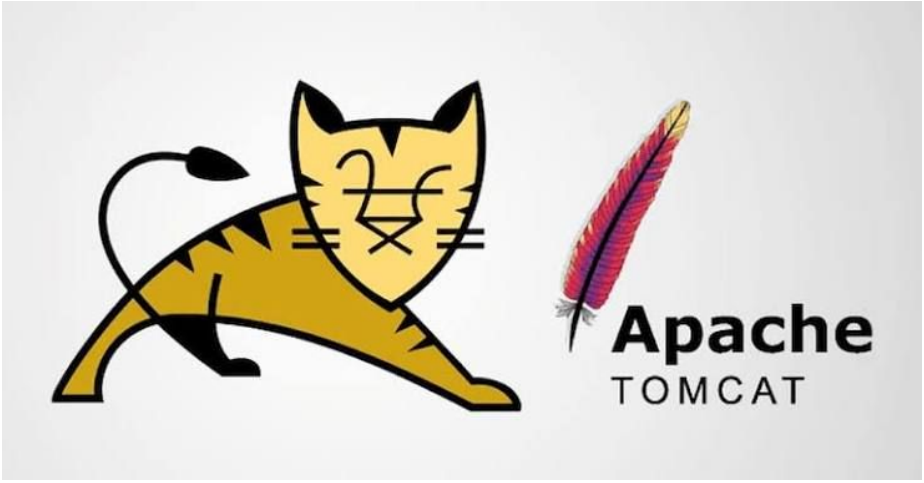





Servlet Container (Tomcat) & Servlet Lifecycle

- **Servlet adalah kelas Java khusus yang menangani HTTP Request dan menghasilkan HTTP Response.**
- **Servlet membutuhkan Servlet Container (seperti Apache Tomcat) untuk mengelola koneksi jaringan dan menjalankan Servlet.**
- **Siklus Hidup (Lifecycle) Servlet dikelola dalam tiga tahap:**
  - `init()`: dipanggil satu kali saat Servlet pertama kali dimuat ke memori untuk inisialisasi awal.
  - `service()` / `doGet()` / `doPost()`: dipanggil setiap kali ada request HTTP masuk dari client.
  - `destroy()`: dipanggil saat web server dimatikan untuk melepaskan sumber daya memori.

https://tomcat.apache.org/








Servlet Pertama dengan Anotasi @WebServlet

- **@WebServlet("/url-path") memetakan jalur URL web langsung ke kelas Servlet penanggung jawab, tanpa perlu mengonfigurasi web.xml.**
  - @WebServlet("/hello") mendaftarkan kelas ini ke URL http://localhost:8080/hello.
  - extends HttpServlet memberikan kemampuan penanganan protokol web pada kelas Java biasa.
  - Method doGet() otomatis tereksekusi saat client melakukan request dengan HTTP Method GET.

```
package com.budiluhur.catalog.servlet;

import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request,
            HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html");
        response.getWriter().println(
            "<h1>Hello from Java Web Servlet!</h1>");
    }
}
```






Hands-On Lab - Web Servlet Pertama

- **Tujuan Lab:**

1. Menambahkan library Servlet & Embedded Tomcat ke project Maven `catalog-product`.
2. Membuat class `ProductServlet` untuk menampilkan daftar produk di browser.
3. Menjalankan aplikasi web menggunakan Embedded Tomcat dan mengujinya di browser.

- **Starting Point: gunakan project Maven `catalog-product` dari Week 2 (sudah memiliki `Product.java`, `ProductNotFoundException.java`, dan `ProductRepository.java`).**






Step 1: Dependency Servlet & Tomcat

- **Buka file pom.xml pada project catalog-product, lalu tambahkan dependency berikut di dalam tag <dependencies>.**
  - Jakarta Servlet API - standar penulisan Servlet di Java modern.
  - Embedded Apache Tomcat - menjalankan server tanpa instalasi Tomcat terpisah.

```
<dependencies>
    <!-- Jakarta Servlet API -->
    <dependency>
        <groupId>jakarta.servlet</groupId>
        <artifactId>
            jakarta.servlet-api
        </artifactId>
        <version>6.0.0</version>
        <scope>provided</scope>
    </dependency>

    <!-- Embedded Apache Tomcat -->
    <dependency>
        <groupId>
            org.apache.tomcat.embed
        </groupId>
        <artifactId>
            tomcat-embed-core
        </artifactId>
        <version>10.1.18</version>
    </dependency>
</dependencies>
```






Step 2: Membuat ProductServlet.java (1/2)

```
package com.budiluhur.catalog.servlet;

import com.budiluhur.catalog.model.Product;
import com.budiluhur.catalog.repository
    .ProductRepository;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http
    .HttpServletRequest;
import jakarta.servlet.http
    .HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.List;

@WebServlet("/products")
public class ProductServlet
        extends HttpServlet {
    private ProductRepository
        productRepository;

@Override
public void init() {
    // Inisialisasi Repository dan data awal
    productRepository =
        new ProductRepository();
    productRepository.addProduct(new Product(
        "PRD-01", "Keyboard Mechanical", 450000.0));
    productRepository.addProduct(new Product(
        "PRD-02", "Mouse Wireless Silent", 175000.0));
    productRepository.addProduct(new Product(
        "PRD-03", "Monitor Gaming 24 Inch", 2100000.0));
}

@Override
protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws IOException {
```






Step 2: Membuat ProductServlet.java (2/2)

```
response.setContentType("text/html");
response.setCharacterEncoding("UTF-8");
PrintWriter out = response.getWriter();
List<Product> products =
    productRepository.findAll();

out.println("<html>");
out.println("<head><title>Katalog "
    + "Produk Web</title></head>");
out.println("<body>");
out.println("<h2>=== DAFTAR KATALOG "
    + "PRODUK (WEB) ===</h2>");
out.println("<table border='1' "
    + "cellpadding='8'>");
out.println("<tr><th>ID Produk</th>"
    + "<th>Nama Produk</th><th>Harga "
    + "Satuan</th></tr>");

for (Product p : products) {
    out.println("<tr>");
    out.println("<td>" + p.getId()
        + "</td>");
    out.println("<td>" + p.getName()
        + "</td>");
    out.println("<td>Rp" + p.getPrice()
        + "</td>");
    out.println("</tr>");
}

out.println("</table>");
out.println("</body>");
out.println("</html>");
    }
}
```






Step 3: App.java (Embedded Tomcat)

```
package com.budiluhur.catalog;

import com.budiluhur.catalog.servlet
    .ProductServlet;
import org.apache.catalina.Context;
import org.apache.catalina.startup.Tomcat;
import java.io.File;

public class App{
    public static void main(
            String[] args)
            throws Exception {
        System.out.println(
            "Menjalankan Embedded "
            + "Tomcat Server...");
        Tomcat tomcat = new Tomcat();
        tomcat.setPort(8080);
        tomcat.getConnector();
        Context ctx = tomcat.addContext("",
            new File(".").getAbsolutePath());
        Tomcat.addServlet(ctx, "ProductServlet",
            new ProductServlet());
        ctx.addServletMappingDecoded(
            "/products", "ProductServlet");
        System.out.println("Server berhasil "
            + "berjalan di: "
            + "http://localhost:8080/products");
        tomcat.start();
        tomcat.getServer().await();
    }
}
```

Step 4: Menjalankan & Menguji Aplikasi Web

- **Langkah pengujian:**

1. Jalankan App.java dari IDE Anda.
2. Buka Web Browser (Chrome/Edge), lalu akses URL: `http://localhost:8080/products`.
3. Browser akan menampilkan tabel HTML berisi data dari ProductRepository.








Mini Challenge

- **Modifikasi ProductServlet.java:**

1. Ambil query parameter `id` via `request.getParameter("id")`.
2. Jika pengguna mengakses `http://localhost:8080/products?id=PRD-01`, tampilkan detail produk dari `productRepository.findById(id)`.
3. Jika produk tidak ditemukan (menangkap `ProductNotFoundException`), set status respon menjadi 404 (`response.setStatus(HttpServletResponse.SC_NOT_FOUND)`) dan tampilkan pesan error HTML berwarna merah.






Common Mistakes

- Error 404 Not Found: disebabkan penulisan URL di browser tidak cocok dengan path pada anotasi `@WebServlet` (bersifat case-sensitive).
- Error 500 Internal Server Error (`NullPointerException`): terjadi karena lupa inisialisasi `productRepository = new ProductRepository();` di dalam method `init()`.
- Browser menampilkan teks kode HTML mentah: terjadi karena lupa menambahkan `response.setContentType("text/html");` sebelum memanggil `response.getWriter()`.
- Perubahan kode Java di IDE tidak memperbarui tampilan browser: disebabkan proses Embedded Tomcat lama belum dimatikan (stop) sebelum dieksekusi ulang (restart).






Recap & Persiapan Minggu Depan

- **Recap:**
  - Protokol HTTP mengatur transaksi Client-Server melalui pesan Request dan Response.
  - HTTP Method utama: GET (baca), POST (tambah), PUT (edit), DELETE (hapus).
  - Status Code HTTP: 200 (sukses), 404 (resource tidak ada), 500 (error server).
  - Servlet adalah komponen server Java yang dikelola oleh Servlet Container (`Tomcat`).
  - Lifecycle Servlet meliputi `init()`, `service()`/`doGet()`, dan `destroy()`.
  - `@WebServlet("/path")` memetakan URL web langsung ke kelas Servlet.

- **Persiapan Minggu ke-4:**
  - Masuk ke Spring Framework: IoC (Inversion of Control) & Dependency Injection.
  - Mempelajari bagaimana Spring Boot mengotomatisasi konfigurasi Tomcat dan Servlet ini agar aplikasi web enterprise dapat dibangun lebih cepat dan terstruktur.






Assignment

- **Tugas individu:**

1. Buat Servlet baru `CategoryServlet.java` yang dipetakan ke URL `/categories`.
2. Buat kelas `Category` (id, name) dan simpan 3 data kategori ke dalam `List<Category>`.
3. Tampilkan daftar kategori dalam format Unordered List HTML (`<ul><li>...</li></ul>`) saat client mengakses `http://localhost:8080/categories`.
4. Kumpulkan screenshot hasil tampilan browser beserta file .java ke portal e-learning atau repo github masing-masing



SELESAI